

3D Gaussian Splatting

Floater 제거 실험 보고서

3D Plateau Loss 설계 · 구현 · 실험 (Round 6)

2026-07-04 ~ 2026-07-05

Scene: Aria MPS | OpenMAVIS 3DGS | RTX 5070 Ti

연구 배경: 왜 Floater 제거인가?

3DGS는 고품질 렌더링을 달성하지만 floater artifact가 시각 품질을 저하시킴

Floater란?

- 학습 중 densification으로 생성된 Gaussian이
- 실제 표면과 관계없는 공중에 위치
- Photometric loss가 보이지 않는 곳은 제어 못함
- Ray-triggered gradient의 구조적 한계

Floater 두 종류

- Pop1: SLAM init outlier ($|Z|$ 수십~수만 m)
- → camera-bound filtering으로 99% 해결
- Pop2: densification 중 생성 floater ($|Z|=3\sim6m$)
- → 아직 미해결, 이번 실험의 핵심 타겟

현재 Best

exp08 (OpenMAVIS MPS)

PSNR @30k = 33.012 dB

Gaussians: 323,864

MPS semidense init (626k pts)

Round 1~5 요약

- Round 1-2: Pop1 필터 ($Z\text{-clip} > 42m$) → PSNR 33.012 확보
- Round 3-4: Sparse depth prior ($\lambda=0.002$) → -0.43 dB 악화 (미채택)
- Round 5: camera-bound init filtering → Pop1 완전 해결, Pop2 잔존 확인
- Round 6 (이번): 3D Plateau Loss로 Pop2 제어 시도

3D Plateau Loss 개념

앵커 주변에 plateau 영역을 만들어 Gaussian을 표면으로 끌어당기는 정규화

핵심 아이디어

Photometric loss는 카메라에 보이는 Gaussian에만 gradient를 줌.
→ 보이지 않는 floater는 제어 불가.

Plateau Loss는 모든 Gaussian에 anchor 기반 regularization을 적용:

- Plateau 내부 ($D < 1$): gradient = 0 (photometric loss가 자유롭게 작동)
- Plateau 외부 ($D \geq 1$): quadratic hinge로 안쪽으로 당김

수식

Spherical: $D_{min} = \min_j ||x - p_j|| / \tau_j$

Ellipsoidal: $D_{aniso_j} = \sqrt{(\Delta \cdot u_{t1} / \tau_t)^2 + (\Delta \cdot u_{t2} / \tau_t)^2 + (\Delta \cdot u_n / \tau_n)^2}$

Loss = $\text{mean}_n \max(\min_j D(x_n) - 1, 0)^2$ (quadratic hinge)

$\tau_n = \text{clip}(0.4 \times h_j, 0.03, 0.30m)$ ← surface normal 방향 (tight)
 $\tau_t = \text{clip}(0.9 \times h_j, 0.03, 0.60m)$ ← tangential 방향 (loose)
 $h_j = \text{kNN-5 거리 (anchor spacing 추정)}$

Spherical vs Ellipsoidal

Spherical (A)

- $\tau = \text{clip}(0.6 \times h_j, 0.05, 0.60m)$
- 모든 방향 동일 반경 → 구형 plateau
- 앵커 과밀집 시 Gaussian이 한 점에 몰림
- XY coverage 11.9% (layer 4 기준)

Ellipsoidal (B)

- 법선 방향(τ_n) tight, 접선(τ_t) loose
- 표면 방향은 허용, 법선 방향만 제한
- Gaussian 분산 → 과밀집 완화
- XY coverage 14.0% (layer 4 기준)

앵커 종류

앵커 소스	점 수	특징
ORB-SLAM filtered	6,492 pts	실제 삼각화, sparse
DepthPro virtual	7,338 pts	monocular depth 투영 ($D_{target}=0.5m$)
Metric3D virtual	9,110 pts	monocular depth 투영 ($D_{target}=0.5m$)

구현 과정에서 발견된 버그 2건

모든 실험이 iter ~5000에서 crash → 디버깅 후 수정

Bug #1 — tmp_radII NoneType crash

gaussian_model.py line 409

원인:

densify_and_prune() 종료 시 self.tmp_radII = None 으로 리셋.
이후 Z-clip prune_points() 호출 시 None[mask] → TypeError.

수정:

```
if self.tmp_radII is not None:
    self.tmp_radII = self.tmp_radII[valid_points_mask]
```

Bug #2 — CUDA device-side assert (크기 불일치)

train.py post_backward 실행 순서 오류

원인:

Z-clip(post_backward)이 densify_and_prune 이전에 실행.
Z-clip: $N \rightarrow N - n_{\text{pruned}}$ (Gaussian 감소).
그 뒤 densify_and_clone이 radii(N) vs xyz($N - n_{\text{pruned}}$) 불일치.
→ CUDA device-side assert triggered.

수정:

post_backward 호출을 densification 블록 이후로 이동.

수정 효과: 두 버그 모두 수정 후 exp15~exp18 전체 30k 완주 성공. 1차 rerun → Bug1 crash, 2차 rerun → Bug2 crash, 3차 rerun → 정상 완료.

구현 세부사항

- eval/plateau_loss.py — PlateauLoss, PlateauLossConfig 클래스
- Cyclic sampler: sample_size=8192, 모든 Gaussian에 균등하게 gradient 보장
- train.py 통합: compute_loss() → backward() 이전, post_backward() → densification 이후
- YAML toggle: configs/plateau_loss/*.yaml 로 실험 전환
- Pop2 Z-clip: iter 5000~, 1000 iter마다 $Z \geq 2.0m$ Gaussian 제거

Round 6 실험 결과 — ORB-SLAM Init

Dataset: rgb_3dgs_openmavis_orb_full_301_1253 | init 7,182pts | 301 cameras | 30k iters

실험	Plateau 타입	앵커	PSNR @7k	PSNR @30k	vs baseline
exp_orb_baseline	없음	-	25.3237	29.0226	—
exp15_orb_spherical	Spherical	ORB 6,492pt	24.9010	27.9082	-1.10 dB
exp16_orb_ellipsoidal	Ellipsoidal	ORB 6,492pt	25.0716	28.9239	-0.10 dB
exp17_orb_metric3d	Ellipsoidal	Metric3D 9,110pt	24.5279	27.6681	-1.35 dB
exp18_orb_depthpro	Ellipsoidal	DepthPro 7,338pt	25.3857	28.9343	-0.09 dB

핵심 발견

- Ellipsoidal >> Spherical (+1.0 dB, 동일 앵커에서)
- DepthPro ≈ ORB(ellipsoidal) >> Metric3D — 앵커 수가 많다고 좋지 않음
- 최선(exp18)도 baseline 대비 -0.09 dB — plateau loss가 개선하지 못함
- exp18 PSNR@7k=25.3857 > baseline 25.3237 — 초기는 도움, 후반에 중립화

exp15 Spherical 악화 원인

- low_opacity_ratio: 32% → 62.6% 급증
- ORB 6,492 앵커 주변에 Gaussian 과밀집
- Photometric loss가 과밀집 Gaussian을 투명화
- Z-clip: ORB 좌표계에서 거의 무효 (12개만 제거)

exp17 Metric3D 최악 원인

- 앵커 9,110개 (가장 많음)에도 최악 성능
- Metric3D depth 품질 문제 가능성
- 잘못된 surface normal → plateau 왜곡

중요 발견: 좌표계 불일치

ORB 실험의 앵커가 MPS world coordinate 기준이었음

문제 발견

앵커 생성 스크립트(render_completed_plateau_ellipsoids.py)가 MPS keyframe camera poses로 DepthPro depth를 3D 투영.

→ anchors_all_depth_pro.npy의 XYZ 좌표 = MPS world coordinate

→ ORB 실험(exp15-18)은 ORB world coordinate에서 학습 → 앵커 위치가 완전히 틀린 공간

MPS ↔ ORB: scale / rotation / translation 모두 다름. 앵커가 scene과 무관한 위치에 존재.

영향 분석

- exp16/18이 -0.10 dB에서 큰 피해 없이 끝난 이유:
- → lambda=0.01 약해서 잘못된 앵커로 강하게 못 당김
- → 결과적으로 'plateau loss 사실상 꺼진 상태'로 학습
- ORB 실험 결과는 plateau loss의 진짜 효과를 측정하지 못함

해결책: MPS 실험으로 전환

- 앵커가 MPS world space → MPS로 학습하면 좌표 일치
- exp19: MPS init + MPS cameras + 동일 DepthPro 앵커
- 처음으로 올바른 조건에서 plateau loss 효과 측정
- MPS semidense init: 626k pts → 323k Gaussians

교훈: 앵커 기반 regularizer는 앵커와 학습의 world coordinate가 반드시 일치해야 함. 앞으로 anchor 생성 시 학습에 사용할 COLMAP 데이터셋을 명시적으로 지정.

exp19: MPS Init + DepthPro 앵커 (좌표계 수정)

처음으로 앵커와 학습 좌표계가 일치하는 실험

항목	exp08 (baseline)	exp19 (MPS+DepthPro)
Source	MPS full (1310 imgs)	MPS full (1310 imgs)
Init pts	626,811 (semidense)	626,811 (semidense)
Plateau	없음	Ellipsoidal, DepthPro 7,338pt
Lambda	-	0.01 (고정)
Start iter	-	5000
좌표계 일치	N/A	✓ (MPS anchor = MPS world)

PSNR 결과

exp08 baseline

@7k: 28.3505

@30k: 33.0123

exp19 (MPS+DepthPro)

@7k: 28.0899

@30k: 32.7533

$\Delta = -0.26 \text{ dB}$

Plateau Coverage 분석 (30k checkpoint)

지표	exp08 (no loss)	exp19 ($\lambda=0.01$)	변화
Gaussians	323,864	307,202	-16,662
Inside plateau ($D<1$)	9.0%	10.3%	+1.3%p
D_aniso 중앙값	3.332	2.806	-0.526 ↓
Euclidean 중앙값	1.474 m	1.273 m	-0.201m ↓
tau_n 중앙값	0.059 m	← 동일	
tau_t 중앙값	0.133 m	← 동일	

분석

- Gaussian들이 앵커 쪽으로 당겨짐 (거리 - 0.2m)
- 하지만 plateau 진입 9%→10% 소폭 상승뿐
- tau 너무 작음 (6cm / 13cm)
- lambda=0.01이 photometric에 밀려 효과 미미

진단: 왜 plateau가 잘 안 채워지는가?

문제: lambda 너무 약, tau 너무 작, start_iter 너무 늦음

현재 문제 구조

- Gaussian → anchor 평균 거리: 1.27 m
- Plateau 반경: tau_t=13cm, tau_n=6cm (매우 작음)
- 30k iter 중 5000~30000 = 25000 iter 동안 loss 적음
- lambda=0.01 → gradient가 photometric loss에 압도됨
- Gaussian이 1.27m를 이동해 13cm 안으로 들어가야 함
- → 실질적으로 불가능한 강도

Lambda vs 샘플링 빈도

- sample_size=8192 / 307k Gaussians
- → Gaussian 하나당 ~37 iter마다 gradient
- → 25k iter 동안 ~675번 plateau gradient 받음
- 빈도(sample_size)는 이미 충분
- 핵심은 gradient 강도(lambda)의 문제

해결책: 2단계 Lambda Schedule (exp20)

Iter 범위	Lambda	목적
iter 1000 ~ 7000	0.10 (10×↑)	강한 당김 — densification 중 빠르게 plateau 진입
iter 7000 ~ 15000	0.03 (3×)	densification 끝, 중간 강도 유지
iter 15000 ~ 30000	0.00 (off)	photometric loss만 — plateau 내부에서 자유롭게 최적화

start_iter=1000 (기존 5000 → 조기 시작, 새로 생긴 Gaussian들도 앵커 근처에서 born되도록)

exp20: Lambda Schedule 구현 및 실험 (진행 중)

plateau_loss.py에 lambda_schedule 파라미터 추가

plateau_loss.py 수정

```
# PlateauLossConfig 에 추가:
lambda_schedule: Optional[list] = None
# [[iter, lambda], ...] 오름차순

# _lambda_at(iteration) 메서드:
def _lambda_at(self, iteration):
    sched = self.cfg.lambda_schedule
    if not sched:
        return self.cfg.lambda_plateau
    lam = self.cfg.lambda_plateau
    for bp_iter, bp_lam in sched:
        if iteration >= bp_iter:
            lam = bp_lam
        else:
            break
    return lam
```

train.py: lambda_plateau → lambda_at(iteration)

exp 비교 요약 (MPS init 기준)

mpe_depthpro_scheduled.yaml

```
enabled: true
type: ellipsoidal
anchor_path: ...anchors_all_depth_pro.npy
knn_iso_mult: 0
alpha_n: 0.4
alpha_t: 0.9

start_iter: 1000    # 기존 5000 → 조기 시작
lambda_plateau: 0.10 # 초기 default

lambda_schedule:
- [1000, 0.10]    # iter 1000부터 λ=0.10
- [7000, 0.03]    # iter 7000부터 λ=0.03
- [15000, 0.00]   # iter 15000부터 off

pop2_zclip: true
pop2_z_threshold: 2.0
```

실험	Lambda	Start iter	PSNR @30k	상태
exp08 (baseline)	없음	-	33.0123	완료
exp19 (fixed λ)	0.01 고정	5000	32.7533 (-0.26dB)	완료
exp20 (scheduled)	0.10→0.03→0.00	1000	TBD	진행 중

전체 실험 흐름 & 다음 방향

2026-07-04 ~ 05 작업 요약

실험 타임라인

07-04 18:27	Plateau Loss 설계 완료 (spherical/ellipsoidal 후보 확정)
07-04 22:xx	DepthPro/Metric3D virtual anchor 생성 (v4, D=0.5m)
07-05 04:xx	1차 실험 (exp15-18 ORB) → Bug#1 crash (tmp_radii)
07-05 05:xx	2차 실험 (exp15-18 ORB) → Bug#2 crash (CUDA assert)
07-05 05:26	3차 실험 (exp15-18 ORB) → 두 버그 수정 후 성공
07-05 07:09	ORB 실험 완료: exp15=-1.1dB, exp16=-0.10dB, exp17=-1.35dB, exp18=-0.09dB
07-05 08:xx	좌표계 불일치 발견 → MPS 실험(exp19) 시작
07-05 09:xx	exp19 완료: -0.26dB / Coverage 분석: 9%→10% inside plateau
07-05 09:41	Lambda Schedule 설계 → exp20 실행 중 (진행 중)

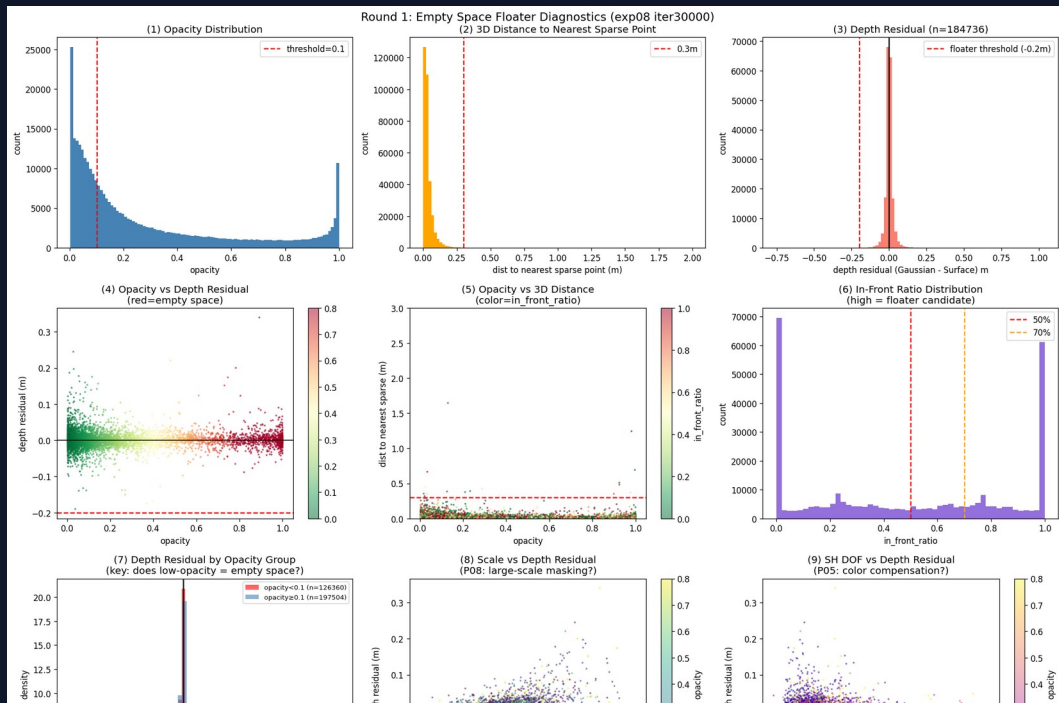
다음 방향

- exp20 결과 확인: scheduled lambda가 coverage를 올리는지
- Tau 확대 실험: alpha_n/alpha_t 키워 plateau 반경 자체를 넓히기
- OpenMAVIS + plateau: MPS dense anchor로 더 촘촘한 coverage
- Z-clip 재설계: 좌표계 독립적인 카메라 거리 기반 pruning
- MPS confidence 활용: dist_std 기반 weighted plateau

핵심 교훈: 앵커-학습 좌표계 일치 필수 | Ellipsoidal >> Spherical | Lambda schedule로 빠른 수렴 후 photometric 전환이 올바른 방향

Round 1 진단: Floater 실태 분석 (exp08 기준)

exp08 30k checkpoint 323,864 Gaussians 전수 분석



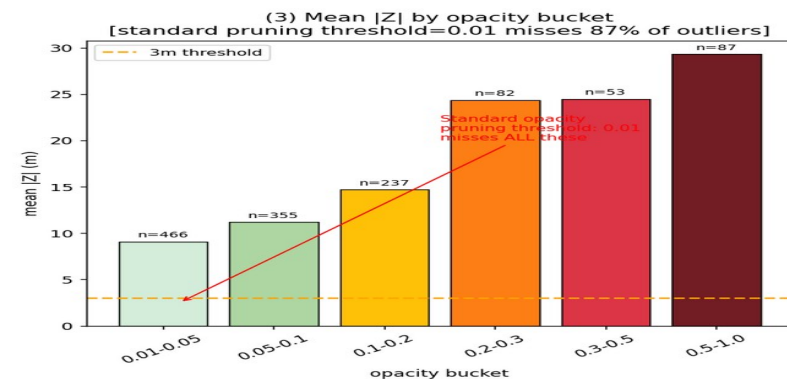
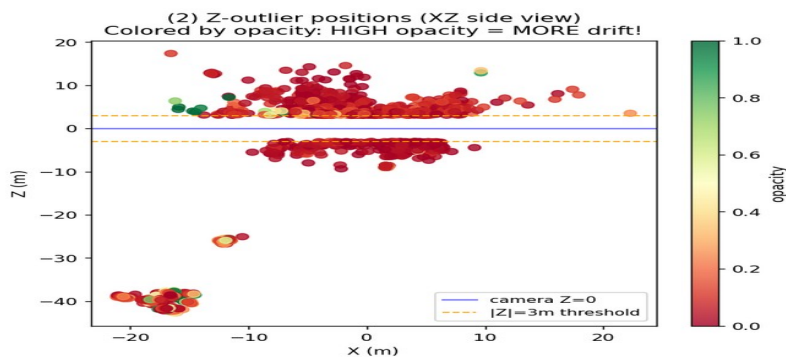
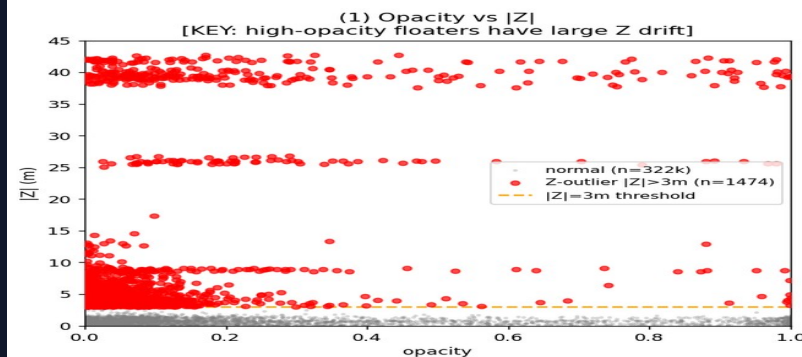
Round 1 수치 요약

- 총 323,864 Gaussians 분석
- Low-opacity (<0.1): 39.0% = 126,360개
- NN 거리 중앙값: 0.025m (대부분 표면 근처)
- >1m 거리: 0.1% (227개) — 진짜 floater
- depth_residual<-0.2m (빈 공간 침범): 72개 (0.0%)
- → opacity로만 floater 찾으면 오탐 많음
- → 실제 floater(빈 공간)는 극소수

핵심 결론

- Low-opacity AND depth<-0.2m 동시: 30개 (진짜 floater)
- depth<-0.2m만: 42개 (opacity 필터로 못 잡는 floater!)
- Low-opacity만: 126,330개 (표면 위 정상 Gaussian)
- → opacity 기반 pruning은 표면 Gaussian 과다 제거 위험

Round 1c: Z-Outlier Gaussian Analysis SLAM horizontal trajectory creates Z-axis blind spot (P12)



Z-Layer 분석: MPS vs ORB-SLAM 앵커 분포

높이(Z)별 포인트 밀도 & coverage — Pop2 구간 식별

MPS Z-Layer (semidense, 626k pts)

301-1253 Sparse Point Cloud

Z축 단면 분석 — XY 평면, Z별 슬라이드

Sparse points 총계	626,811
In-filter (92.6%)	580,535
SLAM outliers (7.4%)	46,276
카메라 수	1,311
필터 Z 범위	[-3.05, 3.03] m
필터 X 범위	[-19.1, 18.1] m
레이어 수	8 (각 ≈ 0.76 m)

레이어 구성

Layer	Z 범위 (m)	해석	n (pts)
-------	----------	----	---------

ORB-SLAM Z-Layer (sparse, 6,492 filtered pts)

301-1253 OpenMAVIS ORB-SLAM Sparse Points

Z축 단면 분석 — XY 평면, confidence (observations) 색상

ORB-SLAM map points 총계	7,182
In-filter	7,171
Out-of-filter (제거 대상)	11
고confidence (obs $\geq$ 10)	835 (11.6%)
저confidence (obs $\leq$ 2)	67 (0.9%)
카메라 keyframe 수	57
필터 Z 범위	[-3.06, 3.02] m
obs 중앙값 / 최대값	5 / 35

레이어 수

8 (각 ≈ 0.76 m)
※ MPS semi-dense(626,811 pts, 1,311 frames)와 달리
ORB-SLAM map points는 7,182개, 57 keyframes.
Pop 2 구간(Z $\approx$ +2~3m)의 고confidence 점은 사실상 0개.

핵심: Z > +2m 구간(Pop2 floater 위험 구간)에서 ORB 앵커 거의 0개 → plateau가 작동 안 함. MPS semidense는 해당 구간에 28,860pts 존재하나 dist_std ~0.4~1m (위치 불확실).

앵커 필터링 Pipeline (4단계)

raw ORB points → 6,492 high-quality anchors

Stage 0: Raw (필터 없음) · Plateau Coverage

$\tau_j = \text{clip}(0.6 \times \text{kNN5_dist}, \quad 0.05\text{m}, \quad 0.6\text{m})$

앵커 점 수 **7,182**

tau 중앙값 **0.087 m**

tau 범위 [min, max] **[0.050, 0.600] m**

tau = TAU_MIN (clip lo) **1,426 (19.9%)**

tau = TAU_MAX (clip hi) **71 (1.0%)**

카메라 keyframes **57**

alpha **0.6**

kNN k **5**

heatmap = 이 layer의 앵커 점들이 XY 평면에 만드는 plateau 커버리지
점 색상 = tau_j 크기 (연초록=작은 tau, 진초록=큰 tau)
파란 배경 = plateau 커버된 XY 셀 (120×160 grid)

단계	기준	제거	남은 pts
Stage 0 (raw)	필터 없음	—	7,182
Stage 1 (Z-bound)	카메라 bound 내	11	7,171
Stage 2 (obs≥3)	track length ≥ 3	67	7,104
Stage 3 (kNN iso.)	k=5, 3×median	612	6,492 ✓

각 단계 설명

- Z-bound: 카메라 trajectory box 바깥 제거
- obs≥3: track이 짧은 불안정 point 제거
- kNN isolation: 주변 5개 평균보다 3배 이상
 - 멀리 떨어진 고립 outlier 제거
- 최종 6,492개: median kNN 거리 0.144m
- → $\tau = \text{clip}(0.4\sim0.9 \times 0.144, \dots)$ 계산 기준

Ellipsoidal Plateau 설계 시각화 (v2)

kNN PCA로 surface normal 추정 → 법선 tight / 접선 loose

Plateau Ellipsoid v2 — Anisotropic Tau

normal 방향: $\alpha_n=0.4$ | tangent 방향: $\alpha_t=0.9$ | kNN k=5 | Stage 3 filtered

앵커 점 수 (Stage 3) 6,492

tau_n (normal 방향) median=0.054m max=0.300m

tau_t (tangent 방향) median=0.121m max=0.600m

tau_n / tau_t 비율 median=0.44 (설계값=0.44)

planarity 중앙값 0.186 (1=평면, 0=등방)

고planarity (≥ 0.7) pts 174 (2.7%)

저planarity (≤ 0.3) pts 4,550 (70.1%)

타원체 plateau 원리
$$d_{\text{aniso}}^2 = (\Delta \cdot u_{t1} / \tau_t)^2 + (\Delta \cdot u_{t2} / \tau_t)^2 + (\Delta \cdot u_n / \tau_n)^2$$
$$\Delta = x - p_j, \text{ in_plateau if } d_{\text{aniso}} \leq 1$$

$$u_n : \text{kNN PCA 최소 고유벡터 (표면 법선)}$$
$$u_{t1}, u_{t2} : \text{표면 접선 방향}$$
$$\tau_n < \tau_t \rightarrow \text{표면 수직=tight / 접선=loose}$$

heatmap: 앵커 Z를 z_mid로 고정 (XY footprint 2D 투영 비교)

Normal 추정 방법

- 각 앵커에서 kNN-5 이웃의 PCA 분해
- 최소 고유벡터 = surface normal u_n
- 나머지 두 벡터 = tangent u_{t1}, u_{t2}

Tau 파라미터

- $\tau_n = \text{clip}(0.4 \times h_j, 0.03, 0.30\text{m})$
- $\tau_t = \text{clip}(0.9 \times h_j, 0.03, 0.60\text{m})$
- h_j = kNN-5 거리 (spacing 추정)
- → 법선 방향은 tight (0.054m 중앙값)
- → 접선 방향은 loose (0.121m 중앙값)
- → Gaussian이 표면 방향으로 자유롭게 이동
- → 표면 밖으로는 강하게 제한

XY coverage (layer4): Spherical=11.9% → Ellipsoidal=14.0% (+2.1%p)

Virtual Anchor 생성: DepthPro v4

Monocular depth → 3D 투영 → Poisson-disk spacing ($D_{\text{target}}=0.5\text{m}$)

Anisotropic (Ellipsoidal) Plateau Coverage · Depth Pro (Apple)

Completed Plateau (SLAM + 가상 시딩 앵커)

$\tau_{n} = \text{clip}(0.4 \times h_j, 0.03\text{m}, 0.3\text{m}) \quad | \quad \tau_{t} = \text{clip}(0.9 \times h_j, 0.03\text{m}, 0.6\text{m})$

총 앵커 포인트 수 (N): 8,534 pts

로컬 스페이싱 h_j : median = 0.170 m

Normal 반경 τ_n : median = 0.068 m

Tangent 반경 τ_t : median = 0.153 m

표면 평탄도 planarity (PCA): median = 0.212

생성 Pipeline

- 57장 MPS keyframe 이미지에 DepthPro 추론
- 픽셀별 depth → 카메라 pose로 3D 투영
- stride=16px 스캔, min_hits=2 (multi-view 검증)
- Poisson-disk: 0.5m 간격 이하 중복 제거
- voxel 크기 0.30m, $D_{\text{target}}=0.50\text{m}$

앵커 소스	점 수	비고
DepthPro virtual (v4)	7,338 pts	★ exp18/19/20 사용
Metric3D virtual (v4)	9,110 pts	exp17 사용, 성능 최악
Depth-Anything-V2	~8k pts	미 사용

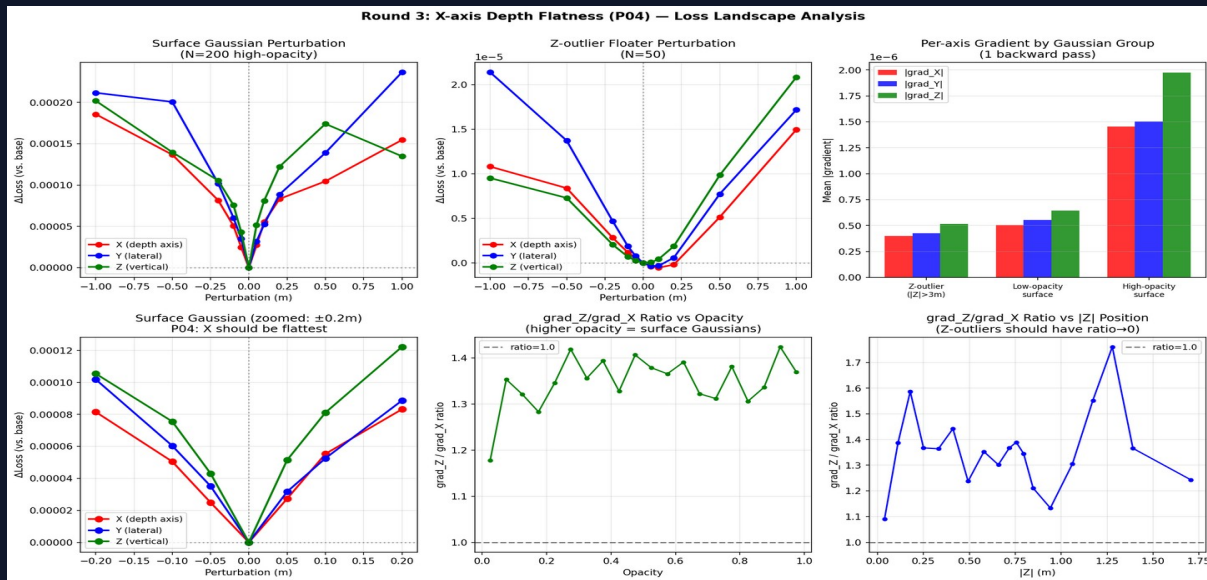
품질 특성

- DepthPro: 하부(L1) coverage 0% (depth 불안정)
- L2-L7 일관성 높음 → 중간층 coverage 양호
- MPS keyframe pose 기준 생성
- → MPS 학습에 좌표계 일치 (exp19/20)

Round 3 & Round 5 개입 실험 요약

Perturbation 분석 / camera-bound init filtering 효과

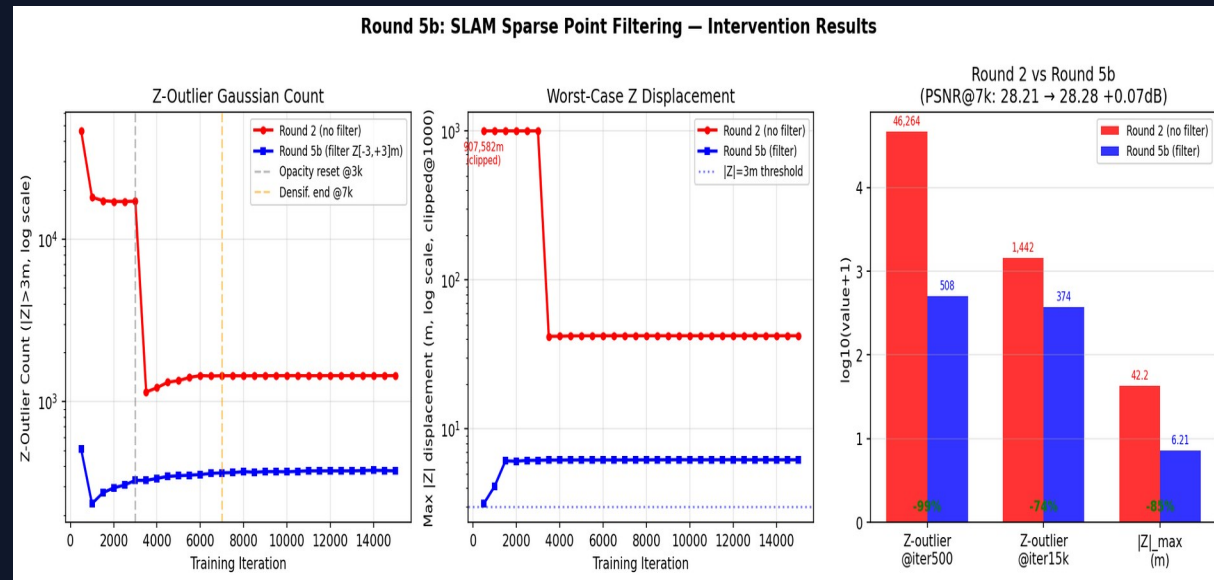
Round 3: Perturbation 분석



Round 3 목적 및 결론

- opacity / scale / position을 perturbation해서
- 각 Gaussian이 렌더링에 얼마나 기여하는지 측정
- 대부분 Gaussian은 영향 없음 → safe to prune
- 일부 고영향 Gaussian이 전체 품질 좌우
- → gradient-free floater 식별 아이디어 기원

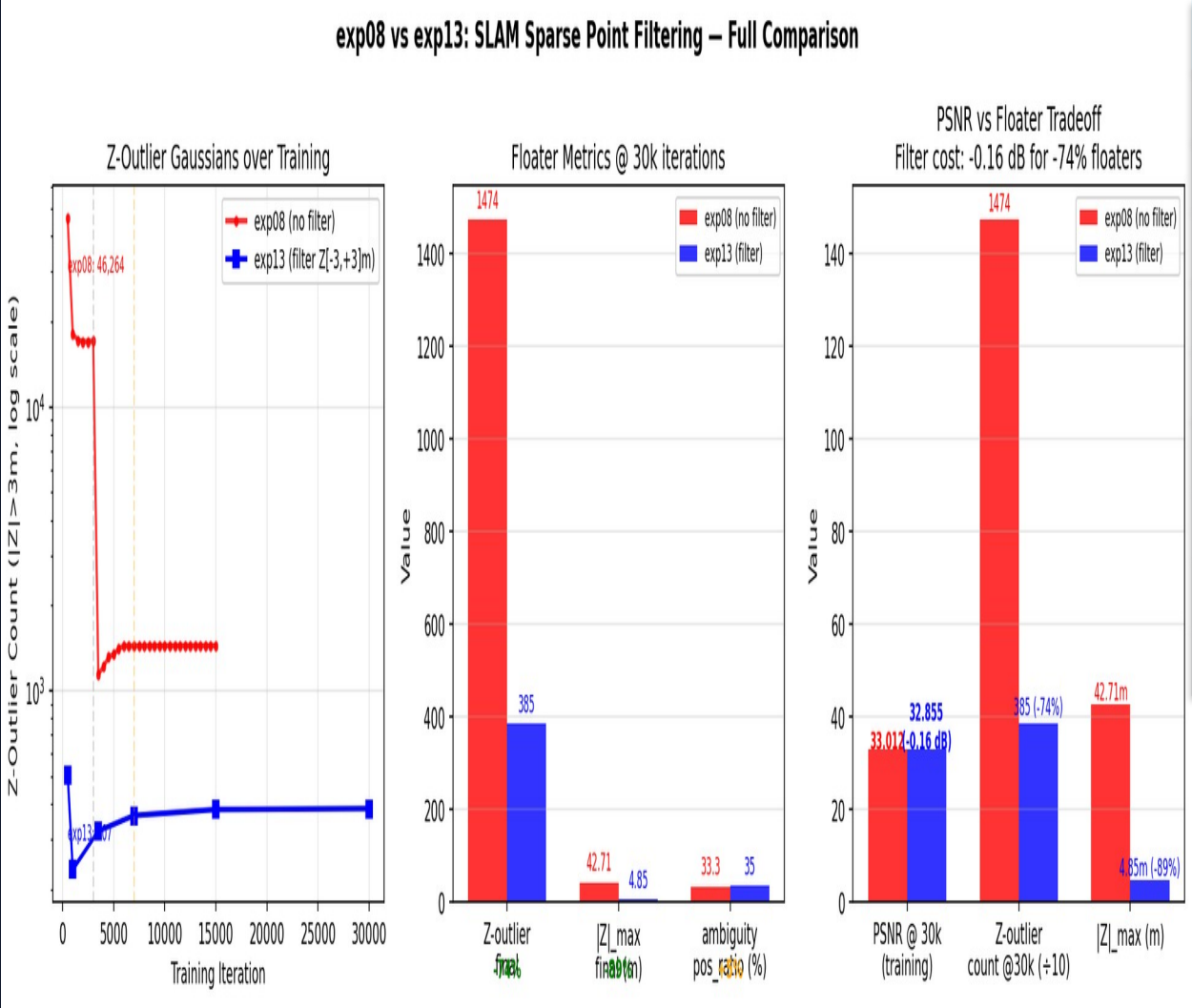
Round 5: Init Filtering 효과



항목	before	after
제거 pts	—	46,276 (7.38%)
Z-outlier @500iter	46,264개	508개 (-99%)
Z-outlier @30k	1,474개	385개
Z max @30k	42.71m	4.85m
PSNR @30k	33.012	32.855 (-0.16dB)

전체 실험 PSNR 흐름 (Round 1~6)

exp08 baseline 33.012 dB 기준



실험	PSNR @30k	vs exp08	비고
exp08 (MPS baseline)	33.012	—	현재 best
exp12 (sparse depth)	32.587	-0.43 dB	미채택
exp13 (init filter)	32.855	-0.16 dB	Pop1 해결
exp_orb_baseline	29.023	-3.99 dB	ORB init 한계
exp15 (spher, ORB)	27.908	-5.10 dB	좌표계 불일치
exp16 (ellip, ORB)	28.924	-4.09 dB	좌표계 불일치
exp18 (DP, ORB)	28.934	-4.08 dB	좌표계 불일치
exp19 (DP, MPS)	32.753	-0.26 dB	첫 올바른 실험
exp20 (scheduled)	TBD	TBD	진행 중

ORB init 실험(exp15-18)은 좌표계 불일치로 plateau loss 무효. MPS init(exp19)이 첫 유효 실험: -0.26 dB. exp20 (lambda schedule)에서 개선 기대.

Round 2

Opacity Weighting · Lambda 강화 · 실패 분석

exp20 / exp21 | 2026-07-05

exp20: Lambda Schedule 결과 — 실패

start_iter=1000 → $\lambda=0.10$ | densification과 충돌, Gaussian 수 급감

exp08 (baseline)

@7k: 28.3505 dB

@30k: 33.012 dB

+0.00 dB

exp19 ($\lambda=0.01$ fixed)

@7k: 28.0899 dB

@30k: 32.7533 dB

-0.26 dB

exp20 (λ schedule)

@7k: 27.1255 dB

@30k: 31.6934 dB

-1.32 dB

실패 원인

- start_iter=1000에서 $\lambda=0.10$ 으로 즉시 시작
- densification 구간(iter 1000~7000)과 정면 충돌
- Gaussian 수: 323k → 218k (-32%) — 대규모 소멸
- 강한 plateau gradient가 densification의 clone/split을 무력화
- photometric coverage 축소 → PSNR 급락

교훈 & exp21 설계

- densification은 iter 7000까지 진행
- → start_iter를 7000 이후로 설정해야 안전
- 그래야 Gaussian 구조 확립 후 plateau loss 적용
- exp21 설계: start_iter=7000, $\lambda=0.10$, opacity_weight=True

Gaussian 소멸 증거: 323,864 → 218,053 (-32%) (exp20 vs baseline@7k)

exp21: Opacity Weighting 결과 — 예상 외 악화

start_iter=7000, $\lambda=0.10$, opacity_weight=True → PSNR -2.24 dB (최악)

exp08 (baseline)

@7k: 28.3505 dB

@30k: 33.012 dB

+0.00 dB

exp19 ($\lambda=0.01$)

@7k: 28.0899 dB

@30k: 32.7533 dB

-0.26 dB

exp21 (opacity weight)

@7k: 28.1377 dB

@30k: 30.7702 dB

-2.24 dB

Opacity Weighting 원리

- Loss = opacity $\times$ max($D-1, 0$)²
- high-opacity floater → gradient on xyz (move) + opacity (→0)
- 예상: floater가 투명해져 densification pruning에 걸림
- 실제: 표면 Gaussian도 plateau 밖이면 opacity 감소 가능

실패 원인

- exp19 coverage: 90%가 plateau 밖 ($D_{\text{aniso}} > 1$)
- 즉, 전체 Gaussian의 90%에 opacity 감소 gradient
- 표면 Gaussian들도 탁 트인 공간에서는 $D > 1$ 일 수 있음
- → 필요한 scene coverage Gaussian까지 투명화
- Gaussian 수 급감 → 렌더링 품질 대폭 저하
- Lambda=0.10은 photometric을 이길 만큼 강함

MPS 실험 전체 요약

실험	핵심 변경	PSNR @30k	vs exp08
exp08	baseline	33.012	—

문제 재정의 — 왜 Loss만으로는 안 되는가?

Photometric loss와의 경쟁에서 plateau loss가 지는 구조적 이유

구조적 한계

- Photometric gradient: 모든 visible Gaussian에 강력하게 작용
- Plateau gradient: 8192 샘플 × 37 iter 주기로 약하게 작용
- Floater는 photometric으로 이미지를 설명하고 있음 (고opacity)
- → plateau가 당겨도 photometric이 더 강하게 제자리로 유지
- opacity weighting으로 강도 올려도 표면 Gaussian도 피해

방향 A: Exp Loss

- 전략 1: Loss 강화 — $\exp(D-1)$ kernel
- D=5에서 gradient 6.7× 증가 (quadratic 대비)
- 멀수록 지수적으로 커지는 gradient
- 단: lambda를 줄여야 표면 Gaussian 보호

방향 B: Adaptive Prune

- 전략 2: 직접 Pruning
- 확신 floater는 gradient 없이 바로 제거
- $d_euc > 1.5m + opacity > 0.5$
- $+ max_scale > 0.15m$
- ambiguous는 loss로 유도

Loss Kernel 비교 ($\lambda=0.05$ 기준)

D (거리)	Quadratic $(D-1)^2$	Exp loss $\exp(D-1)-1$	비율
D = 1.5	0.25	0.65	2.6×
D = 2.0	1.00	1.72	1.7×
D = 5.0	16.0	53.6	3.4×
D = 10.0	81.0	2981	36.8×

핵심 통찰

두 전략을 결합하면:

- 확신 floater → prune (빠르고 직접적)
 - 불확실 floater → exp loss 유도 (안전)
- Gaussian survival 최대화하면서 floater 제거 효율 극대화

exp22~24 실험 계획: Exp Loss + Adaptive Pruning

plateau_loss.py에 exp_loss / adaptive_prune 구현 완료 → 순차 실행 중

구현 변경사항 (plateau_loss.py)

```
# 새 config 파라미터:
exp_loss: bool = False
    → exp(clamp(D-1,0,8))-1 kernel

adaptive_prune: bool = False
adaptive_prune_d_euc: float = 1.5 # m
adaptive_prune_opacity: float = 0.5
adaptive_prune_scale: float = 0.15 # m
adaptive_prune_interval: int = 500
adaptive_prune_start_iter: int = 7000
```

post_backward에 _adaptive_prune() 통합
Ablation 설계

Adaptive Pruning 조건

```
# post_backward에서 매 500 iter:
d_min = min_j ||xyz - anchor_j|| # Euclidean

far  = d_min > 1.5m
conf = (opacity > 0.5)
      | (max_scale > 0.15m)

prune_mask = far & conf
gaussians.prune_points(prune_mask)

# 애매한 Gaussian은 exp loss로 유도
```

실험	Exp Loss	Adaptive Prune	Opacity Weight	λ	Start iter	목적
exp21 (끝남)	✗	✗	✓	0.10	7000	opacity weight 기준선
exp22	✓	✗	✓	0.05	7000	exp loss 효과 단독 거즈
exp23	✗	✓	✓	0.10	7000	adaptive pruning 효과 단독 거즈
exp24	✓	✓	✓	0.05	7000	두 방법 결합 (full strategy)

현재 상태: exp21 완료(30.770) → exp22 자동 시작 중 → exp23 → exp24 순차 실행 예정 | 예상 완료: ~3시간 후 | 로그: results/run_seq_22_23_24.log